In this example, the server will be running on localhost, so use mkcert to generate a key pair and signed certificate if you don't already have one, by running[2]

```
mkcert -pkcs12 localhost
```

in the root folder of the project. You can then implement the DTLS server as shown in listing 12.6. Just as in the client example, you can use the `DtlsDatagramChannel` class to simplify the handshake. Behind the scenes the same handshake process will occur, but the order of `wrap()` and `unwrap()` operations will be different due to the different roles played in the handshake. Open the DtlsServer.java file you created earlier and add the `main` method shown in the listing.

> **NOTE** The `DtlsDatagramChannel` provided in the GitHub repository accompanying the book will automatically connect the underlying `DatagramChannel` to the first client that it receives a packet from and discard packets from other clients until that client disconnects.

Listing 12.6 The DTLS server

```
public static void main(String... args) throws Exception {
    try (var channel = new
DtlsDatagramChannel(getServerContext())) {    #A
        channel.bind(54321);    #A
        logger.info("Listening on port 54321");

        var buffer = ByteBuffer.allocate(2048);    #B

        while (true) {
            channel.receive(buffer);    #C
```